Actividad N4: Cadenas y Punteros

Sofia Salaberry, Martina Araujo, Thiago Goya

Participantes:
Sofia Salaberry, Martina Araujo y Thiago Goya

Escuela:
Escuela Técnica N°1 de Vicente Lopez

Curso:
Ciclo Superior 5°3°

Docentes a cargo:
Yamil Ganduglia, York Elias Mansilla Muñoz

Materia:
Lab. Programación

Repositorio:
Sofia: https://github.com/sopita122/cadenas_y_punteros.git

Año: 2026

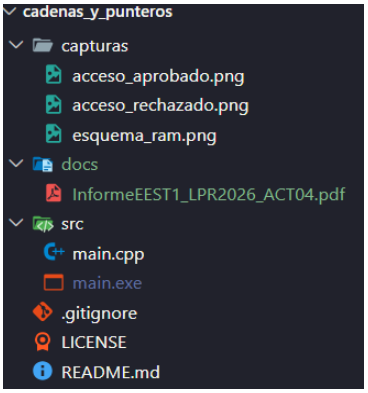
Actividad N4: Cadenas y Punteros

Sofia Salaberry, Martina Araujo, Thiago Goya

Introducción

En este informe se presenta un programa realizado en C + + que permite controlar el acceso de un usuario según su edad, utilizando punteros con el objetivo principal es comprender cómo un puntero puede almacenar la dirección de memoria de una variable y permitir modificar o consultar su valor directamente.

Estructura de Carpetas



Codigo Fuente



```
/*
    ============================================================
    Materia: Laboratorio de Programación (LPR)
    E.E.S.T. Nº 1 "Eduardo Ader" – Vicente López
    Curso: 5º Año 3ª División
    Profesor: Prof. York & Prof. Yamil
    Archivo: main.cpp
    Objetivo: Control de accesos operando directamente sobre la RAM con punteros.
    ============================================================
*/

#include <iostream>

using namespace std;

int main() {
    int edad = 0;
    int* p = &edad; // El puntero 'p' guarda la dirección de memoria física de 'edad'

    // ============================================================
    // REGLA OBLIGATORIA (EVITA EL PLAGIO):
    // Modifiquen la salida agregando su Nombre y Apellido reales.
    // ============================================================
    cout << "=======================================================" << endl;
    cout << "   CONTROL DE ACCESO RAM - ESTUDIANTE: Sofia Salaberry " << endl;
    cout << "=======================================================" << endl;

    cout << "=> Ingrese su edad: ";

    // ============================================================
    // 1. CAPTURA MEDIANTE DESREFERENCIACIÓN (CÓDIGO EXPLICADO)
    // 💡 EXPLICACIÓN TÉCNICA:
    //    Usamos 'cin >> *p;' porque el asterisco '*' desreferencia al puntero.
    //    Esto le indica al compilador que debe guardar el número ingresado
    //    directamente en el casillero de memoria original de 'edad'.
    //    Si pusiéramos 'cin >> p;' sin asterisco, intentaríamos sobrescribir
    //    la dirección hexadecimal física de la RAM, provocando un error grave.
    // ============================================================

    cin >> *p; // Captura el dato guardándolo directo en la memoria apuntada


    cout << "\n--- ANALIZANDO ACCESO SEGURO EN MEMORIA ---" << endl;

    // ============================================================
    // 2. CONDICIONAL CON PUNTERO DESREFERENCIADO
    // 💡 EXPLICACIÓN TÉCNICA:
    //    Evaluamos el valor alojado en la memoria usando '*p >= 18'.
    //    Si mostramos solo 'p', la consola imprimirá la dirección hexadecimal (ej: 0x61fe1c).
    // ============================================================
```

Actividad N4: Cadenas y Punteros

Sofia Salaberry, Martina Araujo, Thiago Goya


```
if (*p >= 18) {
    cout << "[ACCESO APROBADO] El usuario es mayor de edad." << endl;
    cout << "Edad registrada: " << *p << " anos." << endl;
    cout << "Direccion fisica en RAM Hexadecimal: " << p << endl;
} else {
    cout << "[ACCESO RECHAZADO] Menor de edad." << endl;
    cout << "Edad registrada: " << *p << " anos." << endl;
    cout << "Direccion fisica en RAM Hexadecimal: " << p << endl;
}

cout << "======================================================" << endl;
return 0; // Finalización exitosa de Windows (Código 0)
```

Ejecucion de codigo:

Acceso aprobado

```
PS C:\Users\sofia\OneDrive\Escritorio\cadenas_y_punteros\
} ; if ($?) { .\main }
======================================================
   CONTROL DE ACCESO RAM - ESTUDIANTE: Sofia Salaberry
======================================================
=> Ingrese su edad: 20

--- ANALIZANDO ACCESO SEGURO EN MEMORIA ---
[ACCESO APROBADO] El usuario es mayor de edad.
Edad registrada: 20 anos.
Direccion fisica en RAM Hexadecimal: 0x5853dff9c4
======================================================
```

Acceso Negado

```
PS C:\Users\sofia\OneDrive\Escritorio\cadenas_y_punteros> cd "c:\U
======================================================
   CONTROL DE ACCESO RAM - ESTUDIANTE: Sofia Salaberry
======================================================
=> Ingrese su edad: 9

--- ANALIZANDO ACCESO SEGURO EN MEMORIA ---
[ACCESO RECHAZADO] Menor de edad.
Edad registrada: 9 anos.
Direccion fisica en RAM Hexadecimal: 0x76d01ff974
======================================================
```

Actividad N4: Cadenas y Punteros

Sofia Salaberry, Martina Araujo, Thiago Goya

📋 CUESTIONARIO TÉCNICO DE CONTROL

Respondan las siguientes 3 preguntas dentro del informe PDF final que entregarán en Moodle.

🔬 Pregunta 1: Diferencia entre & y *

- **Explicación:** El operador ampersand (&edad) obtiene la ubicación física en la RAM, mientras que el asterisco (*p) accede al contenido de esa dirección.
- **Pregunta:** ¿Qué valor imprime en pantalla la instrucción cout << p; comparada con cout << *p;?

🔬 Pregunta 2: Error Crítico al Guardar Entrada

- **Explicación:** Intentar escribir cin >> p; sin el asterisco es un error grave de compilación porque se intentaría sobrescribir la dirección de memoria física asignada al puntero.
- **Pregunta:** ¿Por qué es obligatorio desreferenciar el puntero (*p) al usar cin para modificar el valor de la variable original?

🔬 Pregunta 3: Formato Hexadecimal

- **Explicación:** Las direcciones de memoria RAM se muestran en formato hexadecimal (base 16) antecedidas por 0x (ej: 0x61fe1c).
- **Pregunta:** ¿Por qué el sistema operativo utiliza la notación hexadecimal en lugar de decimal para representar las direcciones de memoria?

Respuestas

1. Al usar “p”, se muestra la dirección de memoria donde está guardada la variable edad, en cambio, cuando usamos “*p”, mostramos el valor que está guardado en esa dirección, por ejemplo, “p” puede mostrar un valor como 0x61fe1c, mientras que “*p” mostraría la edad que ingresamos.

2. Es obligatorio usar “*p” porque “p” guarda la dirección de memoria de la variable edad, al poner “cin >> *p”, el número que ingresamos se guarda en edad a través del puntero, si ponemos “cin >> p”, estaríamos intentando guardar el dato en el puntero y no en la variable original.

3. Las direcciones de memoria se muestran en hexadecimal porque es una forma más corta y sencilla de representar números grandes, además, el hexadecimal se relaciona fácilmente con la forma en que la computadora trabaja con los bits y la memoria.